SpeechRhythmExtractor

User Manual — SuppMat_SpeechRhythmExtractor.praat

Silva Jr., L. & Barbosa, P. A. (2019–2026)

https://github.com/leonidasjr/SpeechRhythmCode

Supplementary Material

0. About this document

This manual accompanies **SuppMat_SpeechRhythmExtractor.praat**, the version submitted as supplementary material to Cadernos de Linguística (CadLin). It is intended for researchers in linguistics and phonetics who have no prior experience with programming or speech processing software. Every section and line range of the script is documented below, aligned to the script that will appear in the repository.

1. What the script does

SuppMat_SpeechRhythmExtractor.praat is a Praat script that automatically processes all .wav audio files — and their matching. TextGrid annotation files — located in the same folder. For each labelled utterance chunk it extracts 34 prosodic-acoustic parameters and writes two tab-separated datasets:

- Inferential dataset (fileOut$): full metadata + 34 parameters — for mixed-effects models (LMEM) where DIALECT is the fixed effect and SPEAKER is the random effect.
- Predictive dataset (fileOut2$): dialect label + 34 parameters — for binary classification / automatic speech recognition experiments.

Additionally, the script writes a plain-text annotation report (SuppMat_speech_annotation_report.txt) with per-file and grand-total counts of vowels, consonants, syllables, pauses, words, chunks, and total audio duration, ready for use in the Methodology section of a research article.

1.1 SPEAKER code derivation

The SPEAKER code is derived automatically from the audio filename (lines 99–109). The dialect prefix is extracted from characters 1–2 and lowercased (e.g. PB → pb, SP → sp). The numeric suffix is read from the last three characters of the filename stem and converted to a zero-padded 2-digit index (e.g. PBAFEM001 → pb01, SPMAS010 → sp10). Each speaker therefore receives exactly the chunks belonging to their own audio file(s), preserving the natural corpus structure.

2. Requirements

2.1 Software

Praat (this script was developed at version 6.1) freely available at https://www.praat.org. Download for your operating system (Windows, macOS, Linux) and follow the standard installation steps.

2.2 Folder structure

All files must be placed in a single folder together with the script:

```
project_folder/
  ├── SuppMat_SpeechRhythmExtractor.praat
  ├── PBAFEM001.wav
  ├── PBAFEM001.TextGrid
  ├── SPAFEM001.wav
  ├── SPAFEM001.TextGrid
  └── ...
```

> *ℹ The .wav and .TextGrid filenames must be identical (e.g. PBAFEM001.wav and PBAFEM001.TextGrid). The script reads metadata from the filename, so the naming convention described below is mandatory.*

3. Filename convention

The script extracts three metadata fields from the audio filename (without extension):

- Characters 1–2 → DIALECT (e.g. PB, SP)
- Characters 4–6 → SEX (e.g. FEM, MAS)
- Last 3 digits → speaker sequential number → SPEAKER code

Example: PBAFEM001 → DIALECT=PB, SEX=FEM, SPEAKER=pb01.

4. TextGrid annotation structure

Each audio file must have a matching TextGrid with at least 5 tiers:

Tier	Suggested name	Labels	Content
1	`V_to_V`	VV, V_to_V	Inter-vocalic (vowel-to-vowel) intervals — treated as phonetic syllables (Barbosa, 2006)
2	`V_C_Pause`	V, C, #, PAUSE, _, sp, <sp>, sil	Vowels (V), consonants (C), and pause intervals (#, PAUSE, _, sp, <sp>, sil).
3	`(original phone tier from MAUS)`	https://clarin.phonetik.uni-muenchen.de/BASWebServices/interface.	Reference phone tier from MAUS (left on purpose) from realignment of speech units
4	`Word`	orthographic form	Word-level intervals — orthographic transcription
5	`Chunk`	CH1, CH2 …	Utterance chunks — each labelled interval produces one row in the output

> *ℹ Pause labels recognized on Tier 2: #, PAUSE, _, sp, <sp>, sil. Intervals shorter than 50 ms are automatically excluded to avoid Praat auto-boundary artefacts.*

5. Running the script

5.1 Opening and running

1. Open Praat.
2. In the top menu, click Praat → Open Praat script…
3. Navigate to the project folder and select SuppMat_SpeechRhythmExtractor.praat.
4. The script editor will open.
5. Click Run → Run (or press Ctrl+R on Windows/Linux, Cmd+R on macOS).

5.2 The input form (lines 6–21)

The dialog shown in Figure 1 will appear. Fill in the fields as described on the table below. The line numbers in parentheses refer to the actual script.

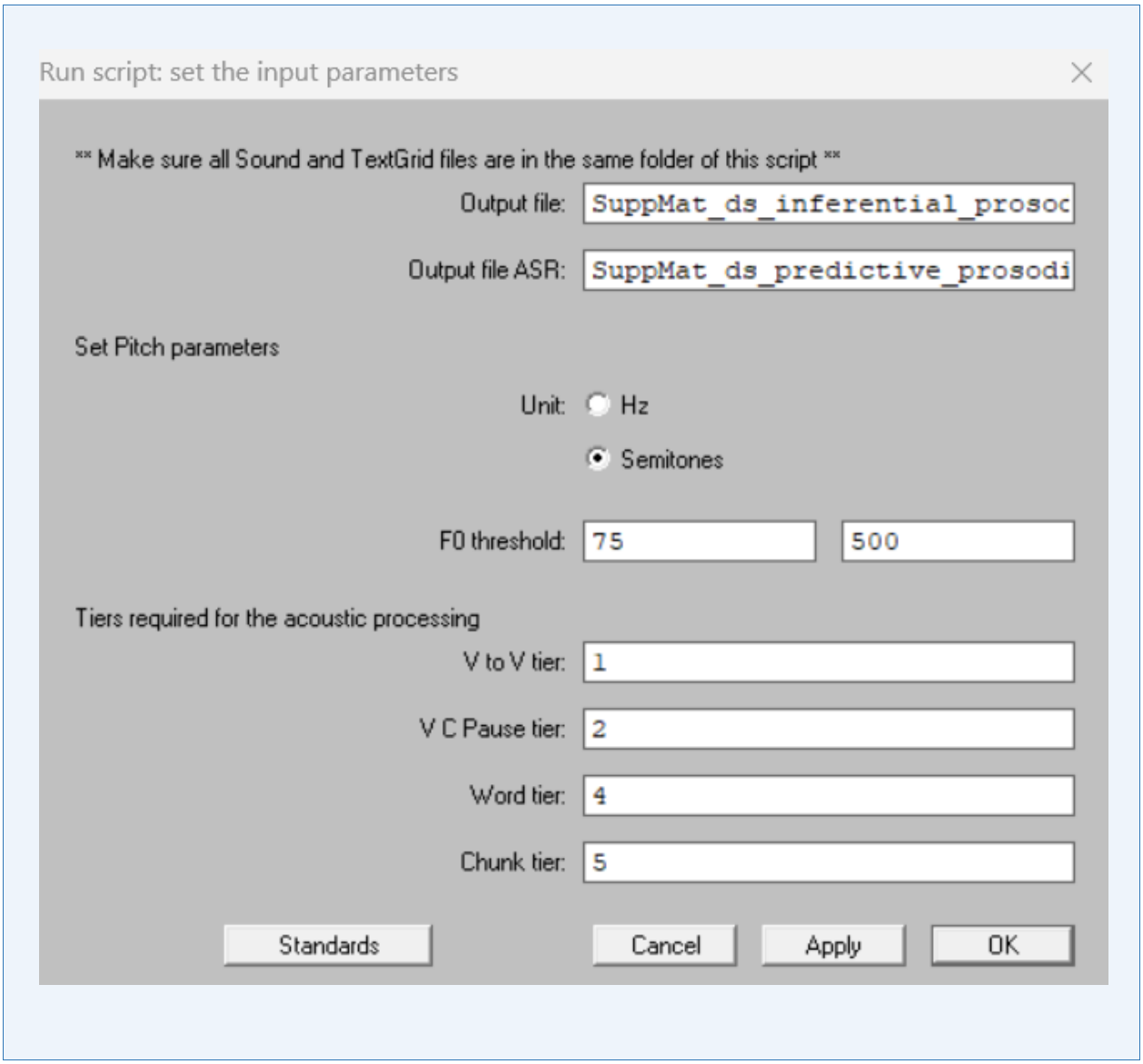

Figure 1. The SpeechRhythmExtractor input form as it appears in Praat when the script is first run. Insert a screenshot of your Praat dialog here.

Field (line)	Default	Type	Description
`Output_file (8)`	`SuppMat_ds_inferential…`	word	Name of the inferential dataset output file (.txt). The script appends .txt automatically.
`Output_file_ASR (9)`	`SuppMat_ds_predictive…`	word	Name of the predictive/ASR dataset output file (.txt).
`Unit (11–13)`	`Hertz / Semitones`	choice	F0 measurement unit. Button 1 = Hz (absolute frequency). Button 2 = Semitones (perceptually scaled, recommended for cross-speaker comparison).
`left_F0_threshold (14)`	`75`	integer	Lower F0 bound in Hz. Typical adults: 60–100 Hz. Lower for very deep voices.
`right_F0_threshold (15)`	`500`	integer	Upper F0 bound in Hz. Typical adults: 400–600 Hz. Higher for children.
`V_to_V_tier (17)`	`1`	natural	TextGrid tier number containing VV (vowel-to-vowel) intervals.
`V_C_Pause_tier (18)`	`2`	natural	TextGrid tier number containing V, C, and pause intervals.
`Word_tier (19)`	`4`	natural	TextGrid tier number containing word intervals.
`Chunk_tier (20)`	`5`	natural	TextGrid tier number containing utterance chunk intervals.

5.3 Monitoring progress

Progress messages are printed in the Praat Info window. On completion, the filenames of all three output files are listed, and the elapsed time is shown (lines 696–708).

6. Output files

6.1 Inferential dataset

Tab-separated .txt file (fileOut$) with columns: AUDIOFILE, DIALECT, SPEAKER, SEX, CHUNK, then all 34 acoustic parameters. For use with mixed-effects models in R (or equivalent), declaring DIALECT as the fixed effect and SPEAKER as the random effect.

6.2 Predictive dataset

Tab-separated .txt file (fileOut2$) with columns: dialect, then all 34 acoustic parameters. No speaker/sex/chunk metadata, to prevent data leakage when training binary classifiers.

6.3 Annotation report

Plain-text file (**SuppMat_speech_annotation_report.txt**) with one row per audio file reporting: TOTAL_DUR_s, CHUNKS, WORDS, SYLLABLES, VOWELS, CONSONANTS, PAUSES, followed by a GRAND TOTALS block. Ready to paste into the Methodology section.

7. Acoustic parameters (34 total)

The table below lists all 34 parameters in the order they appear in the output files. Column 1 (Output column) is the column header in the .txt files; Column 2 (Script variable) is the Praat variable name used in the script.

Output column	Script variable	Description	In practice	Reference
durnorm_Lobanov	durLobanov	Lobanov (1971) z-score of VV syllable durations	Shows how much each syllable deviates from the speaker's own average. +2 = two SD above the speaker's mean = likely emphasis or boundary.	Lobanov (1971); Barbosa (2006)
syl_sd	sylSD	Standard deviation of VV syllable durations (in ms) within the chunk	High values = more stress-timed, expressive pacing. Low values = more syllable-timed, metronomic, even rhythm.	Barbosa (2006); Silva Jr. & Barbosa (2023)
f0norm_Lobanov	f0Lobanov	Lobanov z-score of F0 sampled at f0step intervals	Removes inter-speaker pitch differences. 0 = speaker at their own average pitch.	Lobanov (1971)
f0median	f0median	50th percentile of F0 (semitones or Hz)	Robust center of the pitch range, unaffected by extreme peaks.	Silva Jr. & Barbosa (2023)
f0peak	f0peak	99th percentile of F0	Highest stable pitch event in the chunk — melodic peak.	Silva Jr. & Barbosa (2023)
f0min	f0min	1st percentile of F0	Lowest stable pitch event — melodic valley.	Silva Jr. & Barbosa (2023)
f0sd	f0sd	Standard deviation of F0	How widely pitch varies overall — key correlate of expressiveness.	Silva Jr. & Barbosa (2023)
f0skewness	f0skew	(mean−median)/SD of F0	Positive = more time below mean pitch (falling tendency); negative = rising tendency.	Silva Jr. & Barbosa (2023)
f0SAQ	f0SAQ	(Q3−Q1)/2 of F0 — robust tonal amplitude	Middle-50% pitch range; resistant to outliers.	Silva Jr. & Barbosa (2023)
f0rate	tonerate_total	Total melodic extrema (peaks+valleys) per second	How many pitch turns per second. High = lively intonation.	Barbosa (2020)
f0peak_rate	tonerate_max	F0 peaks per second	Frequency of upward pitch movements — accent density.	Barbosa (2020)
f0min_rate	tonerate_min	F0 valleys per second	Frequency of downward pitch movements.	Barbosa (2020)
f0cv	f0cv	SD / mean of F0	Relative pitch variability: fair comparison across high/low-pitched speakers.	Silva Jr. & Barbosa (2023)
df0mean	meandf0	Mean of F0 discrete first derivative	Average tonal velocity. Near-zero = ascending and descending movements balanced.	Barbosa (2020)
df0mean_pos	meandf0pos	Mean of positive F0 increments	Average speed of rising pitch movements.	Barbosa (2020)
df0mean_neg	meandf0neg	Mean of negative F0 increments	Average speed of falling pitch movements.	Barbosa (2020)
df0sd	sdf0	SD of F0 derivative	Irregularity of tonal velocity — expressive contour = high value.	Barbosa (2020)
df0sd_pos	sdf0pos	SD of positive F0 derivative	Variability of rising movements.	Barbosa (2020)
df0sd_neg	sdf0neg	SD of negative F0 derivative	Variability of falling movements.	Barbosa (2020)

Output column	Script variable	Description	In practice	Reference
df0skewness	skdf0	Skewness of F0 derivative	Positive = more frequent small rises, occasional large falls (continuation pattern).	Barbosa (2020)
spect_emphasis	emphasis	Band energy above vs. below 400 Hz	High = bright/pressed voice; low = soft/breathy voice.	Hammarberg et al. (1980)
sl_LTAS_breath	sl_ltas_high	LTAS slope 0–1 kHz vs 4–8 kHz	Breathiness correlate: steep negative = breathy.	Hammarberg et al. (1980)
sl_LTAS_alpha	sl_ltas_medium	LTAS slope 0–1 kHz vs 1–4 kHz	Alpha-ratio / spectral tilt: negative = modal/pressed.	Hammarberg et al. (1980)
sl_LTAS_L1L0	sl_ltas_low	LTAS slope 300–800 Hz vs 50–300 Hz	L1/L0 spectral ratio; distinguishes modal from non-modal phonation.	Hammarberg et al. (1980)
cvint	cvint	100 × SD / mean of intensity	Loudness variability: high = emphatic speech; low = flat delivery.	Silva Jr. & Barbosa (2023)
jitter	jitter	Local jitter (%) — period perturbation	< ~1% = normal; higher = rough/hoarse voice.	Titze (1995); Boersma (1993)
shimmer	shimmer	Local shimmer (%) — amplitude perturbation	< ~3% = normal; higher = weak/rough voice.	Titze (1995); Boersma (1993)
hnr	hnr	Harmonics-to-Noise Ratio (dB)	Above ~20 dB = clean modal voice; below ~10 dB = noisy/breathy.	Boersma (1993)
pause_sd	deltaPAUSE	SD of pause durations (ms)	High = irregular pausing (unpredictable planning); low = consistent pausing.	Silva Jr. & Barbosa (2023)
pause_meandur	meanDurPAUSE	Mean pause duration (ms)	Typical pause length. Long = deliberate/planning; short = fluent.	Silva Jr. & Barbosa (2023)
pause_rate	pauserate	Pauses / VV duration × 1000	Frequency of pauses relative to speech amount (pauses per second).	Silva Jr. & Barbosa (2023)
speech_rate	srate	1000 / mean VV duration	Overall production speed in syllables per second × 1000.	Barbosa (2006); Silva Jr. & Barbosa (2023)
artic_rate	artrate	Syllables / (VV dur − mean pause) × 1000	True articulated speech production without pausing time.	Barbosa (2020); Silva Jr. & Barbosa (2023)

8. Near-zero fallback values

When a parameter cannot be computed (e.g. no voiced frames for F0, silent chunk for HNR), Praat returns `--undefined--`. The script substitutes a literature-grounded near-zero value so that every chunk produces a complete data row and spurious values are identifiable in downstream analysis.

Parameter(s)	Fallback	Justification (reference)
f0mean, f0median, f0peak, f0min, f0sd	0.001	Avoids log(0) and division by zero in Lobanov normalisation. Strictly positive preserves z = (F0−mean)/SD. (Boersma & Weenink, 2023; Lobanov, 1971)
f0cv, f0skew, f0SAQ, skdf0	0.0	Dimensionless ratios: 0.0 is a valid floor (no variation / perfect symmetry). (Silva Jr. & Barbosa, 2023)
meandf0, meandf0pos, meandf0neg, f0Lobanov	0.0	Zero tonal velocity is phonetically valid in monotone segments. (Barbosa, 2020)

Parameter(s)	Fallback	Justification (reference)
`sdf0, sdf0pos, sdf0neg`	`0.0`	Zero variability of derivative is valid for constant-pitch segments; script guards against n ≤ 1 before sqrt. (Barbosa, 2020)
`tonerate_total, tonerate_max, tonerate_min`	`0.001`	Near-zero (not zero) prevents division by duration in downstream ratios. (Barbosa, 2020)
`sl_ltas_high, sl_ltas_medium, sl_ltas_low`	`-99.0`	Sentinel for failed spectral computation. Well below any natural speech slope. (Hammarberg et al., 1980)
`cvint`	`0.001`	Near-zero prevents division by zero when mean intensity → 0. (Silva Jr. & Barbosa, 2023)
`jitter, shimmer`	`0.0`	Perturbation floor: 0% = perfectly periodic phonation — the theoretical lower bound. (Titze, 1995)
`hnr`	`-20.0`	Fully aperiodic (noise-only) signal. Boersma (1993) confirms reliability range ≈ −20 to +40 dB. (Boersma, 1993)
`pause_sd, pause_meandur, pause_rate`	`0.0`	No pauses = legitimate zero. Guards ensure no division by zero when cptPAUSE = 0. (Silva Jr. & Barbosa, 2023)
`srate, artrate`	`0.001`	Near-zero prevents division by zero when no syllables are detected. (Barbosa, 2006)
`durLobanov, sylSD`	`0.0`	z-score = 0 at mean (valid); SD = 0 for single-syllable chunk (no variation possible). (Lobanov, 1971)

9. Line-by-line code walkthrough

The following blocks document each section of the script. Line ranges refer to the numbered lines of **SuppMat_SpeechRhythmExtractor.praat**.

Lines 1–4 — Header / Comment block

Lines 1–4 are comment lines (beginning with #), not executed. They identify the script, the authors, and the copyright years. The ## (double hash) is a stylistic Praat convention for section-level comments.

Lines 6–21 — FORM block — user dialog

Lines 6–21 define the dialog box that appears when the script is launched (form … endform). Each field type determines what the user can enter:

- word (lines 8–9): free text, used for output filenames.
- choice / button (lines 11–13): mutually exclusive options; Unit: 2 selects Semitones by default.
- integer (lines 14–15): whole-number input for the F0 thresholds.
- natural (lines 17–20): positive-integer input for the TextGrid tier numbers.

All values entered here become Praat variables (e.g., output_file$, unit, left_F0_threshold) that are referenced throughout the rest of the script.

Lines 23–28 — Object cleanup

Lines 23–28 remove any objects already open in Praat's Objects window before the workflow begins. The guard (numberOfSelected() > 0) prevents an error if the Objects window is already empty. This is important when re-running the script in the same Praat session.

Lines 30–33 — Internal constants

Lines 30–33 set four fixed parameters that do not appear in the form:

- smooth_F0_threshold = 2: smoothing bandwidth (Hz) applied to the Pitch object via Smooth.
- window = 0.03: a 30 ms analysis window (referenced internally; not currently used in a direct Praat command, reserved for future extension).
- f0step = 0.05: F0 sampling step in seconds (50 ms) used in the discrete-derivative while-loop (lines 395–412).
- spectral_emphasis_threshold = 400: frequency in Hz that divides the low and high bands for the spectral emphasis computation (line 255).

Lines 35–41 — File list & timer

Line 38 scans the current folder for all *.wav files and stores them in a Strings object called audioDataList. Line 39 reads the number of files into numberOfFiles. Line 41 starts the internal stopwatch that will be read at the very end (line 704) to report elapsed processing time.

Lines 43–62 — Annotation report — initialization

Lines 43–62 create the plain-text annotation report file SuppMat_speech_annotation_report.txt (line 46). Line 47 deletes any pre-existing file with that name. Lines 48–53 write the header rows. Line 53 defines the column names of the per-file table: FILE, SPEAKER, DIALECT, TOTAL_DUR_s, CHUNKS, WORDS, SYLLABLES, VOWELS, CONSONANTS, PAUSES.

Lines 56–62 initialize seven grand-total accumulators (grand_dur, grand_chunks, grand_words, grand_syl, grand_vowels, grand_cons, grand_pauses) to zero before the main loop begins.

Lines 64–81 — Output file headers — inferential & predictive datasets

Lines 64–73 build the inferential dataset file (fileOut$). Line 67 constructs the full filename by appending .txt to the name entered in the form. Line 68 deletes any existing file with that name. Lines 69–73 write the tab-separated column heading row across four continuation lines (...): AUDIOFILE, DIALECT, SPEAKER, SEX, CHUNK, followed by all 34 acoustic parameter column names.

Lines 75–81 repeat this procedure for the predictive dataset file (fileOut2$), which begins with dialect and then all 34 acoustic columns (no AUDIOFILE, SPEAKER, SEX, or CHUNK, to prevent data leakage in classification experiments).

Lines 83–90 — Main loop — start message

Lines 83–90 print progress messages to the Praat Info window (writeInfoLine / appendInfoLine) so the user can see that processing has begun. The separator lines (==========) help visually delimit the output.

Lines 91–109 — Outer file loop — metadata extraction & SPEAKER derivation

Lines 91–109 begin the outer loop (for y from 1 to numberOfFiles). For each file:

- Lines 92–95: select the Strings object, retrieve the filename, read the .wav file, and store the Sound object name in soundFile$.
- Line 96: extract language$ (dialect code) from characters 1–2 of the filename (e.g. PB or SP).
- Line 97: extract sex$ from characters 4–6 (e.g. FEM or MAS).
- Lines 99–109 (SPEAKER derivation): the script builds speaker$ from the filename. First, lowerDialect$ converts the uppercase dialect code to lowercase (e.g. PB → pb). Then stemLen = length(soundFile$) measures the filename stem (without extension). numStr$ = mid$(soundFile$, stemLen-2, 3) extracts the last three characters as a string (e.g. "001"), which is converted to an integer via speakerNum = number(numStr$). If speakerNum < 10, a zero is prepended (e.g. pb01); otherwise the number is appended directly (e.g. pb10).

Lines 111–127 — TextGrid loading & per-file counter initialisation

Lines 111–113: the TextGrid filename is built by appending .TextGrid to soundFile$ and then loaded. Line 113 selects the TextGrid object for subsequent tier queries.

Lines 115–126 initialise per-file counters: file_dur (total audio duration), file_chunks, file_words, file_syl, file_vowels, file_cons, file_pauses — all start at zero for each new file.

Lines 128–147 — Inner chunk loop — chunk extraction

Lines 128–129 query the total number of intervals in chunk_tier (Chunk_tier from the form). Lines 131–135 begin the inner loop (for x from 1 to nChunks). For each interval, the script reads start_time, end_time, and chunk$ (the text label). Only intervals with a non-empty label (if chunk$ <> "", line 135) are processed. Line 137 increments file_chunks. Lines 139–146 record the exact boundaries of the labelled chunk (startChunk, endChunk) and extract that portion of the Sound file as a temporary chunk_filename$ object (Extract part).

Lines 148–197 — Word counting & phoneme/syllable counting

Lines 148–174 count words. The script finds the first and last word-tier intervals that overlap with the chunk (word_start, word_end) and iterates them (for w from word_start to word_end). Each non-empty word label increments word_counter. Inside the word loop (lines 161–171) a Pitch object is also created for the word to compute peak_ratio — the average of the 51st, 75th, and 99th percentiles of F0, which feeds the melodic rate calculation.

Lines 176–197 count phoneme-sized units from the V_C_Pause tier: labels V increment file_vowels (line 184), C increment file_cons (line 186). Lines 190–197 count syllables (VV or V_to_V labels) from the V_to_V tier into file_syl.

Lines 199–215 — LTAS computation & undefined guards

Lines 200–204: To Ltas... 100 computes the Long-Term Average Spectrum of the chunk with a 100 Hz resolution. Three slopes are extracted:

- sl_ltas_low (line 202): slope between 300–800 Hz vs 50–300 Hz — L1/L0 ratio.
- sl_ltas_medium (line 203): slope between 0–1 kHz vs 1–4 kHz — alpha-ratio / spectral tilt.
- sl_ltas_high (line 204): slope between 0–1 kHz vs 4–8 kHz — breathiness correlate.

Lines 206–215 apply the near-zero guard: if any slope is undefined (e.g., insufficient signal), it is replaced with −99.0, a sentinel value well below any natural speech slope (Hammarberg et al., 1980).

Lines 217–229 — Intensity — cvint

Lines 217–225: To Intensity... 'left_F0_threshold' 0.0 yes creates an intensity object using left_F0_threshold as the minimum pitch (Hz) parameter, which affects smoothing. Get mean and Get standard deviation extract the mean energy-domain intensity (mint) and its SD (sdint). Lines 221–225 compute cvint = 100 × sdint / mint only when mint > 0, otherwise setting cvint = 0.001 (near-zero guard). Lines 226–229 apply a second guard for the undefined case.

Lines 231–259 — Voice quality — jitter, shimmer, harmonic LTAS, spectral emphasis

Lines 231–237: a PointProcess (periodic, cc) is created, then combined with the chunk Sound to compute an LTAS (only harmonics). The harmonic LTAS yields lowmean (1.4–32 Hz band) and highmean (32–64.3 Hz band), producing sPI = lowmean − highmean (not output directly; an internal voice quality indicator).

Line 239: Get jitter (local) with parameters 0.0 (start), 0.0 (end), 0.0001 (minimum period), 0.02 (maximum period), 1.3 (maximum factor). Line 240 converts jitter to percentage. Lines 241–244 apply the near-zero guard (0.0).

Lines 245–251: plus Sound combines the PointProcess and Sound for Get shimmer (local) with parameters 0 0 0.0001 0.02 1.3 1.6 (the last value is maximum amplitude factor). Line 247 converts to percentage. Lines 248–251 apply the near-zero guard.

Lines 253–259: To Spectrum... yes creates a power spectrum; Get band energy difference computes the difference in log energy between the band 0–spectral_emphasis_threshold Hz and the rest — the spectral emphasis index. Near-zero guard: 0.001.

Lines 261–321 — F0 extraction — Pitch object & derived measures

Lines 261–263 create a new Pitch object from the chunk (To Pitch... 0.0 left_F0_threshold right_F0_threshold) and smooth it. Lines 265–283 branch on unit (2 = Semitones, 1 = Hz) to extract f0mean (mean), f0median (p50), f0peak (p99), f0min (p01), f0sd (SD), and the interquartile bounds f01Q (p25) and f03Q (p75). When unit = 1, Smooth and Interpolate are applied to improve Hz-domain continuity.

Lines 285–306 apply near-zero guards to all F0 values (→ 0.001), preventing log/division failures in downstream normalisation.

Lines 308–321 compute three derived measures and their guards:

- f0cv = f0sd / f0mean (coefficient of variation)
- f0skew = (f0mean − f0median) / f0sd
- f0SAQ = (f03Q − f01Q) / 2

All default to 0.0 if undefined.

Lines 323–341 — F0 peak/valley timing statistics

Lines 323–341 compute statistics on the timing of F0 maxima and minima across the chunk. peak_time (line 323) and valley_time (line 324) use Get time of maximum/minimum with parabolic interpolation in Hertz. The code then accumulates a running sum for mean_peak_time and

sd_peak_time (lines 327–333) and equivalently for valley timing (lines 334–341). These are internal diagnostics for melodic structure analysis; they are not written to the output files.

Lines 343–382 — Melodic rate — PointProcess extrema

Lines 343–347 convert the smoothed Pitch to a Matrix (To Matrix), then to a 1D waveform (To Sound (slice)... 1) and count its extrema (PointProcess). To PointProcess (extrema)... 1 yes no None counts only peaks → ntones → tonerate_max = ntones / totaldur.

Lines 349–361 define the melodic_rate procedure, which re-extracts the chunk sound, computes a new Pitch object, smooths it, converts to Matrix and then Sound slice. This is called twice:

- Line 363–366 (call melodic_rate + yes yes): all extrema (peaks and valleys) → tonerate_total.
- Line 368–371 (call melodic_rate + no yes): valleys only → tonerate_min.

Lines 373–382 apply near-zero guards (0.001) to all three rates.

Lines 384–485 — F0 derivative computation (df0)

Lines 384–412 compute the discrete first derivative of F0. The Pitch object is converted to a PitchTier (Down to PitchTier, line 385). Counters are initialised (meandf0, meandf0pos, meandf0neg, ipos, ineg). The while-loop (lines 395–412) steps through the chunk from startChunk in increments of f0step (0.05 s). At each step:

- f0current = Get value at time (line 396)
- f0Lobanov = (f0current − f0mean) / f0sd (Lobanov z-score, line 397)
- df0'i' = f0current − f0ant (first difference, line 398)
- If df0'i' > 0: accumulated into meandf0pos and stored in df0pos'ipos' array.
- If df0'i' ≤ 0: accumulated into meandf0neg and stored in df0neg'ineg' array.

After the loop (lines 413–434): i is decremented by 1 (correcting the off-by-one from the last i = i+1), means are divided, and f0Lobanov is averaged across all steps. Near-zero guards (0.0) are applied.

Lines 436–475 compute standard deviations (sdf0, sdf0pos, sdf0neg) using $\Sigma(x-\text{mean})^2/(n-1)$, guarded against n ≤ 1 to avoid sqrt of zero/negative.

Lines 477–485 compute the skewness of df0 using the unbiased formula $(i / ((i-1)\times(i-2))) \times \Sigma((x-\text{mean})/sd)^3$, only when sdf0 > 0 and i > 2; otherwise skdf0 = 0.0.

Lines 487–494 — HNR — Harmonicity object

Lines 487–490 compute HNR on the full Sound file (not the chunk extraction) within the chunk time window. To Harmonicity (cc): 0.01, 40, 0.1, 1 creates a Harmonicity object using cross-correlation with parameters: time step 0.01 s, minimum pitch 40 Hz, silence threshold 0.1, periods per window 1. Get mean retrieves the mean HNR in dB within startChunk–endChunk. Lines 491–494 apply the near-zero guard: hnr = −20.0 dB if undefined (Boersma, 1993).

Lines 496–526 — Speech rate & syllable duration (procedure syl_duration)

Lines 497–499 find the first and last VV intervals within the chunk (startVV, nVV). Lines 503–514 define the syl_duration procedure: it iterates intervals on v_to_V_tier from startVV to nVV−1,

collects all VV or V_to_V intervals, stores their durations in the array durVV'cptVV', and accumulates the total sVVdur. Line 515 calls the procedure.

Lines 517–526 compute meanDurVV (average VV duration in ms) and srate = 1000/meanDurVV (speech rate in syllables per second ×1000), both guarded for the case cptVV = 0.

Lines 528–571 — Syllable SD (syl_sd) & Lobanov z-score (durnorm_Lobanov)

Lines 528–540 compute syl_sd: the sum of squared deviations from meanDurVV is accumulated in varVV, then sddurVV = sqrt(varVV/(cptVV−1)) (sample SD). sylSD = sddurVV. If cptVV ≤ 1, both are set to 0.0 (near-zero guard: single-syllable chunks have no variation).

Lines 542–571 compute durLobanov (durnorm_Lobanov). A second VV loop (startVV to nVV−2) computes the z-score for each VV interval: zdurVV = (durVV'cptVV' − meanDurVV)/sddurVV. It also computes znextDurVV for the next interval and accumulates |zdurVV − znextDurVV| into szVV (not output; an internal rhythmic contrast measure). durLobanov stores the z-score of the last VV interval processed. If undefined (e.g. first chunk has only one VV), it defaults to 0.0.

Lines 573–642 — Pause block — mean duration, artrate, pause SD, pause rate

Lines 573–591 count and accumulate pauses. The loop iterates the V_C_Pause tier from startVC to nVC−1. Labels recognised as pauses: #, PAUSE, _, sp, <sp>, sil (line 581). Each matching interval's duration is computed in ms (durPAUSE_ms = (end_pause − start_pause)×1000). Only intervals ≥ 50 ms are counted (line 585), excluding Praat automatic boundary artefacts. Qualifying pauses increment cptPAUSE, store individual durations in durPAUSE'cptPAUSE', and add to sPAUSEdur. Line 592 adds cptPAUSE to file_pauses (for the annotation report).

Lines 594–599: if cptPAUSE > 0, meanDurPAUSE = sPAUSEdur/cptPAUSE; otherwise meanDurPAUSE = 0.0.

Lines 601–614 recount VV intervals (cptVV, sVVdur) for the artrate denominator — this second VV loop operates on the full tier rather than the sub-range used for syl_duration, ensuring consistent scope.

Lines 616–621 compute artrate: if (sVVdur − meanDurPAUSE) > 0, artrate = (cptVV/(sVVdur − meanDurPAUSE))×1000; otherwise artrate = srate (fallback guard).

Lines 623–636 compute deltaPAUSE (pause_sd): if cptPAUSE > 1, the sample SD of durPAUSE'i' is computed; if undefined, deltaPAUSE = meanDurPAUSE; if cptPAUSE ≤ 1, deltaPAUSE = 0.0.

Lines 638–642: pauserate = |cptPAUSE/sVVdur|×1000 if sVVdur > 0, else 0.0.

Lines 644–662 — Write data rows — inferential & predictive datasets

Lines 644–649 write one tab-separated row to fileOut$ (inferential dataset) using fileappend, spanning five continuation lines (...). The values are: filename$, language$ (DIALECT), speaker$, sex$, chunk$ (metadata), then all 34 acoustic parameters. Most are formatted to 2 decimal places (:2); the three LTAS slope values use 1 decimal place (:1).

Lines 651–656 write the equivalent row to fileOut2$ (predictive dataset), starting with language$ (dialect label) and all 34 parameters — no AUDIOFILE, SPEAKER, SEX, or CHUNK columns.

Lines 658–662 clean up all temporary objects created during this chunk iteration (all objects except TextGrid 'soundFile$', Sound 'soundFile$', and Strings audioDataList).

Lines 663–678 — End of chunk & file loops

Line 663 closes the if chunk$ <> "" block (the main chunk processing). Line 665 re-selects the TextGrid so the next iteration of the x-loop can continue querying intervals. Line 667 closes the for x (chunk) loop. Lines 669–676 accumulate per-file totals into grand totals and write one row to the annotation report for this file. Line 678 closes the outer for y (file) loop.

Lines 680–694 — Grand totals & final cleanup

Lines 680–690 append the GRAND TOTALS block to the annotation report: total audio duration (s), total chunks, words, syllables (VV intervals), vowels (V), consonants (C), and pauses (≥50 ms).

Lines 692–694 remove all remaining objects except the audioDataList Strings, then remove that too, leaving the Objects window clean.

Lines 696–708 — Final messages & elapsed time

Lines 696–703 print completion messages to the Info window, listing the three output filenames (inferential dataset, predictive dataset, annotation report) so the user can verify them immediately.

Lines 704–708 read the stopwatch, compute minutes and remaining seconds, round to two decimal places, and print the elapsed time.

10. Troubleshooting

Symptom	Cause and solution
Script stops with 'division by zero'	A chunk has no syllables or no pauses. Near-zero guards (Sections 8 and 9) handle this automatically. Check that your Chunk tier has non-empty labels.
Output file is empty	No .wav files found in the script folder. Ensure SuppMat_SpeechRhythmExtractor.praat is in the same folder as all audio files.
TextGrid not found	The .TextGrid filename **must match** the .wav exactly (case-sensitive on macOS and Linux).
F0 values showing as undefined	F0 is outside the threshold range. Adjust **left_F0_threshold** or **right_F0_threshold** in the form.
Columns appear merged in Excel	Open as tab-delimited, not comma-delimited. In Excel: Data → From Text/CSV → Delimiter = Tab.
Very slow processing	Normal for large speech *corpora*; every chunk undergoes multiple Praat operations. Close other Praat objects before running.

11. Citation

When using this script in publications, please cite:

SILVA JR., L.; BARBOSA, P. A. SpeechRhythmExtractor. [Computer program for Praat], 2019–2026. Available at: https://github.com/leonidasjr/SpeechRhythmCode.

BARBOSA, P. A. ProsodyDescriptorExtractor (Version 2.0). [Computer program for Praat], 2020. Available at: https://github.com/pabarbosa/prosody-scripts.

SILVA JR., L.; BARBOSA, P. A. Voice disguise and foreign accent: Prosodic aspects of English produced by Brazilian Portuguese speakers. Estudios de Fonética Experimental / Journal of Experimental Phonetics, v. 32, p. 195–226, 2023. DOI: https://doi.org/10.1344/efe-2023-32-195-226.

References (ABNT format)

BARBOSA, P. A. **Incursões em torno do ritmo da fala**. Campinas: Pontes Editores, 2006.

BARBOSA, P. A. **ProsodyDescriptorExtractor** (Version 2.0). [Computer program for Praat], 2020. Available at: https://github.com/pabarbosa/prosody-scripts.

BOERSMA, P. Accurate short-term analysis of the fundamental frequency and the harmonics-to-noise ratio of a sampled sound. **Proceedings of the Institute of Phonetic Sciences**, v. 17, p. 97–110, 1993. Available at: https://www.fon.hum.uva.nl/paul/papers/Proceedings_1993.pdf.

BOERSMA, P.; WEENINK, D. **Praat: Doing phonetics by computer**. [Computer program], Version 6.3+. Amsterdam: University of Amsterdam, 1992–2026. Disponível em / Available at: https://www.praat.org.

HAMMARBERG, B.; FRITZELL, B.; GAUFFIN, J.; SUNDBERG, J.; WEDIN, L. Perceptual and acoustic correlates of abnormal voice qualities. **Acta Otolaryngologica**, v. 90, n. 5–6, p. 441–451, 1980. DOI: https://doi.org/10.3109/00016488009131746.

LOBANOV, B. M. Classification of Russian vowels spoken by different speakers. **The Journal of the Acoustical Society of America**, v. 49, n. 2B, p. 606–608, 1971. DOI: https://doi.org/10.1121/1.1912396.

SILVA JR., L.; BARBOSA, P. A. **SpeechRhythmExtractor**. [Computer program for Praat], 2019–2026. Available at: https://github.com/leonidasjr/SpeechRhythmCode.

SILVA JR., L.; BARBOSA, P. A. Voice disguise and foreign accent: Prosodic aspects of English produced by Brazilian Portuguese speakers. **Estudios de Fonética Experimental / Journal of Experimental Phonetics**, v. 32, p. 195–226, 2023. DOI: https://doi.org/10.1344/efe-2023-32-195-226.

TITZE, I. R. **Workshop on Acoustic Voice Analysis: Summary Statement**. Denver: National Center for Voice and Speech, 1995. Available at: https://ncvs.org/archive/freebooks/summary-statement.pdf.

TITZE, I. R.; HORII, Y.; SCHERER, R. C. Some technical considerations in voice perturbation measurements. **Journal of Speech and Hearing Research**, v. 30, n. 2, p. 252–260, 1987. DOI: https://doi.org/10.1044/jshr.3002.252.